

Programming Assignment 3—Iterative Data-Flow Analysis and Client Optimizations

Marc Lin

1 Live Variable Analysis (LVA)

Follow by the 5-step approach in assignment and the LVA definition in lecture slides, we first recognize local variables which are created by users. By doing so, check type and add them to private member *namedVars*.

```
// Step #1: identify program variables.
for (auto &bb : F) {
    for (auto &inst : bb) {
        if (inst.getOpcode() == llvm::Instruction::Alloca) {
            if (inst.hasName()) {
                namedVars.insert(inst.getName());
            }
        }
    }
}
```

In Step 2 we only want *LoadInst* and *StoreInst* because they affect USE and DEF sets. *StoreInst* is to save value to user-defined local variables (DEF), while *LoadInst* is to use value from variables (USE). We can check *namedVars* to ensure we only take user-defined variables.

```
for (auto it = bb.rbegin() ; it != bb.rend() ; it++) {
    // LoadInst
    if (LoadInst* LI = dyn_cast_or_null<LoadInst>(&*it)) {
        if (namedVars.count(LI->getPointerOperand()->getName())) { // found
            USE.insert(LI->getPointerOperand()->getName());
            DEF.erase(LI->getPointerOperand()->getName());
        }
    }
    // StoreInst
    else if (StoreInst* SI = dyn_cast_or_null<StoreInst>(&*it)) {
        if (namedVars.count(SI->getPointerOperand()->getName())) { // found
            USE.erase(SI->getPointerOperand()->getName());
            DEF.insert(SI->getPointerOperand()->getName());
        }
    }
}
```

In Step 3 we simply call *computePostOrder* and use the postorder in step 4. LVA is backward analysis using postorder. We simply implement IN/OUT algorithm

from lecture slides with an additional iteration counter. In step 5 we output Live-in/Live-out information.

2 Dead Code Elimination (DCE)

isDead is a helper function for **instruction-level** dead code check. We can do DCE with helper checking each Instruction is dead or not. Once the dead code is found, we also recursively mark all dependencies by *findDeadDefinitions*. Then we follow Algorithm 1 to sweep all dead code.

```
// 1. Mark step
for (auto &bb : F) {
    for (auto &inst : bb) {
        if (lv.isDead(inst)) {
            deadVars.insert(&inst);
            if (StoreInst *SI = dyn_cast<StoreInst>(&inst)) {
                Value *val = SI->getValueOperand();
                if (Instruction *II = dyn_cast<Instruction>(&inst)) {
                    deadVars.insert(II);
                }
            }
            // recursively find all dependent dead code, add to deadVars
            findDeadDefinitions(&inst, deadVars);
        }
    }
}
```

When doing *eraseFromParent*, since BB does not automatically record the next and prev iterator, we record the next iterator first, then we erase the current one.

```
// Step #2: remove the Alloca instructions having no uses.
// TODO: Implement
for (auto &bb : F) {
    auto it = bb.begin(); // start iterator
    while (it != bb.end()) {
        Instruction &inst = *it++; // next instruction (same with *(it++) )
        if (inst.getOpcode() == llvm::Instruction::Alloca && inst.use_empty()) {
            // need to save next iterator first,
            // because bb will not record ext iterator automatically.
            inst.eraseFromParent();
        }
    }
}
```

3 Available Expressions Analysis (AEA)

Follow by the 4-step approach in assignment and the AEA definition in lecture slides, an instruction is considered an “expression” for this analysis if it is either a *BinaryOperator* or a *CmpInst*.

Note: I use `isa<llvm::type>(&instruction)` instead of using `dyn_cast` in some following cases, because we cannot do multiple `dyn_cast` in one *if-statement*, binary logical operators such as `&&`, `||` do not support such actions.

```
// Step 1: Identify all expressions and create mappings
// PA3: Implement
uint32_t ID = 0;
for (auto &bb : F) {
    for (auto &inst : bb) {
        if (isa<BinaryOperator>(&inst) || isa<CmpInst>(&inst)) { // same with dyn_cast
            expr2Id[&inst] = ID;
            id2Expr.push_back(&inst);
            universalSet.insert(ID);
            ID++;
        }
    }
}
```

When computing K, we need to consider two cases: (1) when *Inst* produces a result value. (2) when *Inst* is the *StoreInst*. These two cases identify which expressions are killed by current instruction.

```
// case 1: when inst produces a result value
if (!inst.getType()->isVoidTy() && !isa<AllocaInst>(&inst)) {
    // example: %result = add i32 %x, %y
    // instruction 'I' defined the Value '%result'
    Instruction *I = &inst; // Access the entire instruction (add)
    Value *result_I = I; // Access the result of operation (%result) by Upcasting
    for (uint32_t id : universalSet) {
        Instruction *expr = id2Expr[id];
        for (User::op_iterator op = expr->op_begin(); op != expr->op_end(); ++op) {
            Value *operandValue = op->get(); // Access actual operand value (llvm::Value)
            // means '%result' is used as some expr's operand, so expr is killed by 'I'
            if (operandValue == result_I) {
                K.insert(id);
            }
        }
    }
}
```

After computing K, find instruction if it's in `expr2Id` (i.e. is expression), then compute G and update GEN and KILL.

```

// Compute G
if (expr2Id.count(&inst)) {    // if it's generated (expr)
    G.insert(expr2Id[&inst]);
}

// Compute GEN, KILL set
GEN = set_union(G, set_difference(GEN, K));
KILL = set_union(K, set_difference(KILL, G));

```

In step 3 and 4 we implement IN/OUT algorithm for AEA, which use intersection as meet operator instead of union. Note I use *pred_begin* and *pred_end* to find predecessors for each basic block.

```

// Compute IN set
set<uint32_t> newIn = {};
bool first = true;
for (auto it = pred_begin(curr), end = pred_end(curr) ; it != end ; it++) {
    if (first) {
        newIn = bb2Out[*it];
        first = false;
    }
    newIn = intersect(newIn, bb2Out[*it]);
}
bb2In[curr] = newIn;

```

4 Common Subexpression Elimination (CSE)

By AEA, we now have IN and OUT for each basic block, and we also implement *isAvailableAfter* for **instruction-level** availability. This is implemented in the same way as AEA, except for doing insert and erase effect on *available* instead of computing K and G. Break if we reach the point, then return if id of target expr is in *available*.

```

if (operandValue == result_I) {
    available.erase(id);    // KILL effect on available
}

```

```

// Apply GEN effect on available
if (expr2Id.count(&inst)) {    // if it's generated (expr)
    available.insert(expr2Id[&inst]);
}

```

In CSE we implement Algorithm 2 and 3. In Mark step, the purpose is to find the common subexpression before *inst*, so we use *Search* to find *prevInst*, then check

availability to see if we can redirect all usage of *inst* to *prevInst*.

```
for (auto &prevInst : available) {    // Instruction *
    if (Search(&inst, prevInst) && &inst != prevInst) {
        BasicBlock::iterator insertionPoint = std::prev(BasicBlock::iterator({
            if (ae.isAvailableAfter(*prevInst, *insertionPoint)) {
                for (auto op = inst.op_begin() ; op != inst.op_end() ; op++) {
                    if (LoadInst *LI = dyn_cast<LoadInst>(op->get())) {
                        DeadLoads.push_back(LI);
                    }
                }
                inst.replaceAllUsesWith(prevInst);
                exprsToDelete.push_back(&inst);
                break;
            }
        }
    }
}
```

As for the Sweep step, sweep all the marked expressions, also check if *LoadInst* is useless now. After calling *eraseFromParent*, check if loop converges (i.e. reach the fixed point).

```
// SWEEP step
for (auto &inst : exprsToDelete) {    // Instruction *
    inst->eraseFromParent();
}

vector<Instruction*> LoadsToDelete;
for (auto &LI : DeadLoads) {          // Instruction *
    if (LI->use_empty()) {
        LoadsToDelete.push_back(LI);
    }
}
for (auto &inst : LoadsToDelete) {
    inst->eraseFromParent();
}

// check if reached fixed-point
if (!exprsToDelete.empty()) {
    changedInIter = true;
}
if (changedInIter) {
    globalChanged = true;
}
else {
    break;
}
```